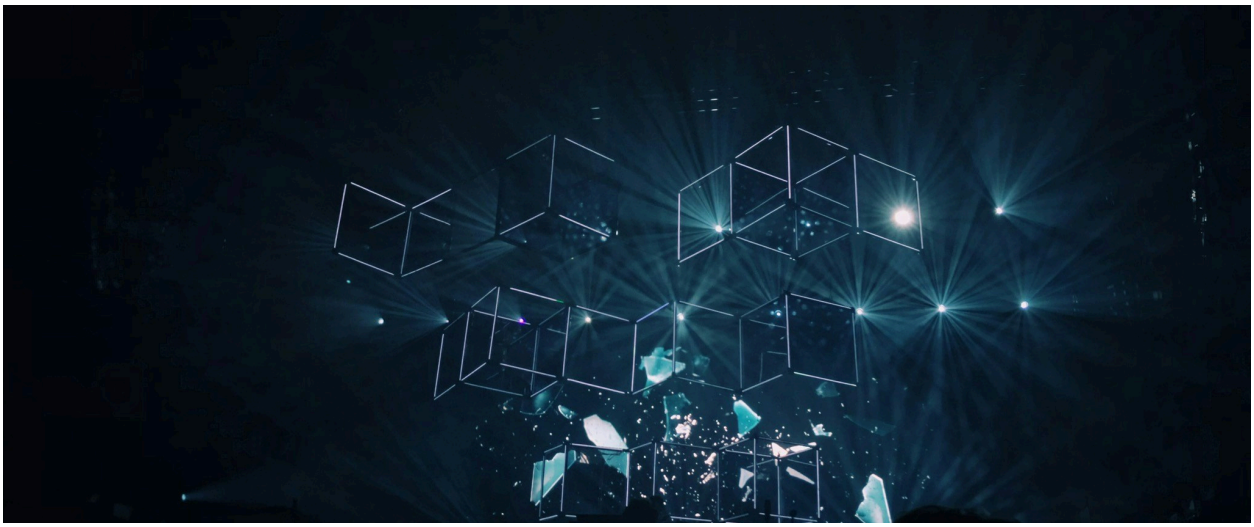

MySQL Joins and Subqueries

by Kranti Patil
Fusion Software Institute

Joins are used to combine data from multiple tables based on related columns, while subqueries allow you to use the result of one query inside another query.

Inner Join

Definition: Returns only the rows that have matching values in both tables.

Syntax

```
SELECT columns
FROM table1
INNER JOIN table2 ON table1.column = table2.column;
```

Example

```
-- Get customer names with their orders
SELECT customers.name, orders.order_date, orders.amount
```

```
FROM customers
INNER JOIN orders ON customers.customer_id = orders.customer_id;
```

Key Points

- Only returns matching records from both tables
- Most commonly used join type
- NULL values in join columns are excluded
- Can join multiple tables by chaining INNER JOINs

Left Join (Left Outer Join)

Definition: Returns all rows from the left table and matching rows from the right table. If no match, NULL values for right table columns.

Syntax

```
SELECT columns
FROM table1
LEFT JOIN table2 ON table1.column = table2.column;
```

Example

```
-- Get all customers and their orders (including customers with no orders)
SELECT customers.name, orders.order_date, orders.amount
FROM customers
LEFT JOIN orders ON customers.customer_id = orders.customer_id;
```

Key Points

- Preserves all records from the left (first) table
- Shows NULL for unmatched records from the right table
- Useful for finding records that don't have matches

- Can use `WHERE right_table.column IS NULL` to find non-matching records

Right Join (Right Outer Join)

Definition: Returns all rows from the right table and matching rows from the left table. If no match, NULL values for left table columns.

Syntax

```
SELECT columns
FROM table1
RIGHT JOIN table2 ON table1.column = table2.column;
```

Example

```
-- Get all orders and customer details (including orders without customer info)
SELECT customers.name, orders.order_date, orders.amount
FROM customers
RIGHT JOIN orders ON customers.customer_id = orders.customer_id;
```

Key Points

- Preserves all records from the right (second) table
- Shows NULL for unmatched records from the left table
- Less commonly used than LEFT JOIN
- Can be rewritten as LEFT JOIN by switching table order

Full Outer Join

Definition: Returns all rows when there's a match in either table. Shows NULL values where no match exists.

> ⚠️ **Important:** MySQL doesn't support FULL OUTER JOIN directly, but you can simulate it using UNION.

Syntax (Simulated)

```
SELECT columns FROM table1 LEFT JOIN table2 ON condition
UNION
SELECT columns FROM table1 RIGHT JOIN table2 ON condition;
```

Example

```
-- Get all customers and all orders (whether matched or not)
SELECT customers.name, orders.order_date, orders.amount
FROM customers
LEFT JOIN orders ON customers.customer_id = orders.customer_id
UNION
SELECT customers.name, orders.order_date, orders.amount
FROM customers
RIGHT JOIN orders ON customers.customer_id = orders.customer_id;
```

Key Points

- Combines results of LEFT and RIGHT joins
- Shows all records from both tables
- NULL values appear where no match exists
- Use UNION to eliminate duplicates, UNION ALL to keep them

Self Join

Definition: A table is joined with itself to compare rows within the same table.

Syntax

```
SELECT columns
FROM table1 t1
JOIN table1 t2 ON t1.column = t2.column;
```

Example

```
-- Find employees and their managers
SELECT e1.name AS employee, e2.name AS manager
FROM employees e1
INNER JOIN employees e2 ON e1.manager_id = e2.employee_id;

-- Find customers from the same city
SELECT c1.name AS customer1, c2.name AS customer2, c1.city
FROM customers c1
INNER JOIN customers c2 ON c1.city = c2.city AND c1.customer_id < c2.custo
mer_id;
```

Key Points

- Requires table aliases to distinguish between instances
- Useful for hierarchical data (employee-manager relationships)
- Can find relationships within the same dataset
- Can use any join type (INNER, LEFT, RIGHT) with self joins

Subqueries with IN

Definition: Returns records where a column value matches any value in a subquery result set.

Syntax

```
SELECT columns
FROM table
WHERE column IN (SELECT column FROM another_table WHERE condition);
```

Examples

```
-- Find customers who have placed orders
SELECT *
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);

-- Find products in specific categories
SELECT *
FROM products
WHERE category_id IN (SELECT category_id FROM categories WHERE name I
N ('Electronics', 'Books'));

-- Using NOT IN to find customers with no orders
SELECT *
FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders WHERE cust
omer_id IS NOT NULL);
```

Key Points

- Equivalent to multiple OR conditions
- Can use NOT IN for exclusion
- Be careful with NULL values in NOT IN queries
- Often can be rewritten as JOINs for better performance

Subqueries with ALL

Definition: Compares a value to ALL values returned by a subquery. The condition must be true for all values.

Syntax

```
SELECT columns
FROM table
```

```
WHERE column operator ALL (SELECT column FROM another_table WHERE c
ondition);
```

Examples

```
-- Find products more expensive than ALL products in the 'Books' category
SELECT *
FROM products
WHERE price > ALL (SELECT price FROM products WHERE category = 'Book
s');

-- Find customers with orders greater than all orders from a specific date
SELECT DISTINCT customer_id
FROM orders o1
WHERE amount > ALL (SELECT amount FROM orders WHERE order_date = '2
024-01-01');

-- Find the highest paid employee in each department
SELECT *
FROM employees e1
WHERE salary >= ALL (SELECT salary FROM employees e2 WHERE e1.depart
ment = e2.department);
```

Common Operators with ALL

- `> ALL`: Greater than the maximum value
- `< ALL`: Less than the minimum value
- `= ALL`: Equal to all values (only true if subquery returns one value or all identical values)
- `!= ALL`: Not equal to any value (equivalent to NOT IN)

Subqueries with ANY

Definition: Compares a value to ANY value returned by a subquery. The condition needs to be true for at least one value.

Syntax

```
SELECT columns
FROM table
WHERE column operator ANY (SELECT column FROM another_table WHERE c
ondition);
```

Examples

```
-- Find products cheaper than any product in the 'Electronics' category
SELECT *
FROM products
WHERE price < ANY (SELECT price FROM products WHERE category = 'Electr
onics');

-- Find customers who have at least one order greater than any order from 20
24
SELECT DISTINCT customer_id
FROM orders o1
WHERE amount > ANY (SELECT amount FROM orders WHERE YEAR(order_da
te) = 2024);

-- Find employees earning more than any employee in the 'HR' department
SELECT *
FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department =
'HR');
```

Common Operators with ANY

- `> ANY`: Greater than the minimum value
- `< ANY`: Less than the maximum value

- `= ANY`: Equal to at least one value (equivalent to IN)
- `!= ANY`: Not equal to at least one value

Summary

Join Type	Returns	Use Case
INNER JOIN	Only matching records	Most common, when you need data from both tables
LEFT JOIN	All left + matching right	When you need all records from the primary table
RIGHT JOIN	All right + matching left	Less common, opposite of LEFT JOIN
FULL OUTER JOIN	All records from both tables	When you need everything (simulate with UNION)
SELF JOIN	Related records within same table	Hierarchical data, comparing rows within table

Subquery Operator	Purpose	Example Use
IN	Match any value in set	Find customers with orders
ALL	Compare against all values	Find products more expensive than ALL in category
ANY	Compare against any value	Find products cheaper than ANY in category